

ESERCITAZIONE 8

Corso di
CVeDI (8 CFU)

Elia Guarnieri
elia.guarnieri@unimib.it

Git, open source distributed version control system

1. Cosa è un VCS
2. Git
3. Installare git
4. Git in VSCode
5. Git nel cloud

Cosa è un VCS

Version Control System (VCS)

Un Version Control System (VCS) è uno strumento che **registra e gestisce le modifiche apportate a un insieme di file nel tempo**

L'obiettivo principale di un VCS è **tenere traccia delle variazioni nel codice sorgente** di un progetto software o in qualsiasi altro insieme di file, **consentendo a più persone di collaborare in modo efficiente**

Version Control System (VCS)

Il VCS **registra le modifiche** apportate ai file nel corso del tempo **in un repository**, che funge da deposito centrale di tutte le versioni del progetto

Gli utenti possono **recuperare versioni precedenti**, **confrontare le modifiche** e, in molti casi, **risolvere conflitti** quando più persone modificano lo stesso file contemporaneamente

L'utilizzo di un VCS facilita la gestione delle versioni, la collaborazione e il tracciamento delle modifiche nel corso del ciclo di sviluppo del software o di altri progetti

Tipi di VCS

Esistono due tipi principali di VCS

Sistemi di Controllo Versione Centralizzati (CVCS)
In un CVCS, esiste un repository centralizzato che conserva tutte le versioni dei file. Gli utenti lavorano direttamente su una copia locale dei file, ma devono comunicare con il repository centrale per ottenere gli aggiornamenti più recenti o per registrare le proprie modifiche
Esempio di CVCS: Subversion (SVN)

Sistemi di Controllo Versione Distribuiti (DVCS)
In un DVCS, ogni utente ha una copia completa del repository, inclusi tutti i file e la storia delle modifiche. Questo consente agli utenti di lavorare in modo autonomo, senza la necessità di una connessione continua al repository centrale. Possono eseguire commit, creare branch e fare merge delle modifiche localmente. Quando è necessario sincronizzare con gli altri utenti, possono scambiarsi le modifiche
Esempio di DVCS: Git

Quale scegliere?

Entrambi i tipi di VCS hanno vantaggi e svantaggi: I DVCS offrono maggiore flessibilità e sono più adatti per progetti distribuiti e team remoti; I CVCS sono più centralizzati e seguono un modello più tradizionale, spesso utilizzato in ambienti di sviluppo di grandi dimensioni

La scelta tra i due dipende dalle esigenze specifiche del progetto e del team di sviluppo

Attualmente, **il VCS più diffuso è Git** ed è anche quello che vedremo; tuttavia, è importante notare che ci sono situazioni e contesti in cui i sistemi di controllo delle versioni centralizzati (CVCS), come Subversion (SVN), possono ancora essere utilizzati, soprattutto in organizzazioni che hanno una lunga storia di utilizzo di tali sistemi o in progetti specifici che richiedono un approccio centralizzato

Git

Git

Git è un sistema di controllo delle versioni distribuito (DVCS) progettato per gestire il tracciamento delle modifiche nel codice sorgente durante lo sviluppo del software. Creato da Linus Torvalds nel 2005, Git è diventato uno degli strumenti più utilizzati nell'ambito dello sviluppo software

Git è ampiamente utilizzato nell'industria del software e nella comunità di sviluppatori open source. La sua popolarità è dovuta a diverse caratteristiche

Principali caratteristiche di Git

Distribuito: Ogni sviluppatore ha una copia completa dell'intero repository, consentendo il lavoro in modalità offline e facilitando la collaborazione tra sviluppatori distribuiti

Velocità: Git è noto per la sua velocità nelle operazioni di versionamento e gestione dei rami (branch), il che lo rende adatto per progetti di varie dimensioni

Branching e merging efficienti: Git facilita la creazione, la gestione e il merging di rami (branch), consentendo lo sviluppo parallelo di funzionalità senza compromettere la stabilità del codice principale

Storia completa delle modifiche: Ogni modifica apportata al repository è tracciata in modo dettagliato, fornendo una storia completa delle modifiche nel tempo

Collaborazione: Git supporta efficacemente la collaborazione tra team di sviluppatori, consentendo loro di lavorare in modo coordinato su diverse parti di un progetto

Open source: Git è un software open source con una vasta comunità di sviluppatori che contribuiscono al suo miglioramento continuo.

Integrazione con servizi di hosting: Git è spesso utilizzato insieme a servizi di hosting come GitHub, GitLab e Bitbucket per la gestione centralizzata dei repository e la collaborazione online

Flusso di lavoro - concetti chiave

Esistono 4 elementi fondamentali nel flusso di lavoro con Git

1. **La Working Directory**, la cartella del progetto dove sono presenti tutti i file
2. **La Staging Area**, l'area dove sono inseriti i file modificati e pronti per essere salvati nel local repository
3. **Il Local Repository (HEAD)**, la versione locale del repository
4. **Il Remote Repository (REMOTE)**, il repository centralizzato che funge da hub centrale per la collaborazione

Flusso di lavoro - azioni base

Repository: Un repository Git è una raccolta di file e cartelle che rappresenta il progetto. Può essere locale (sulla macchina di uno sviluppatore) o remoto (su un server Git come GitHub)

Clone: Per iniziare a lavorare su un progetto, uno sviluppatore clona il repository. Questo crea una copia locale del repository, consentendo modifiche indipendenti

Commit: Gli sviluppatori effettuano modifiche nei propri file ed eseguono dei *commit* per registrare queste modifiche nel repository locale. Un commit rappresenta una singola unità di modifica

Branch: Git facilita la creazione di rami (branch) per sviluppare funzionalità in modo isolato. Un branch consente di lavorare su nuove idee senza influire sul codice principale

Merge: Dopo aver completato lo sviluppo su un branch, è possibile integrare le modifiche nel branch principale (solitamente chiamato main o master) tramite un merge. Git gestisce automaticamente le fusioni quando possibile. In caso di conflitti, gli sviluppatori li devono risolvere manualmente

Pull e Push: Per aggiornare il repository locale con le modifiche apportate da altri sviluppatori sul repository remoto, si scaricano le modifiche (*pull*). Per condividere le proprie modifiche con il repository remoto, si caricano le modifiche (*push*)

Stato (Status) e Diff: È sempre possibile controllare lo stato attuale delle modifiche non ancora registrate (status) e visualizzare le differenze tra il lavoro corrente e l'ultima modifica registrata (*diff*)

Storia: Git mantiene una storia completa delle modifiche. Gli sviluppatori possono sempre esplorare la storia del progetto (*log*) per visualizzare i commit precedenti ed eventualmente tornare a uno stato precedente del progetto

Flusso di lavoro - stati del file

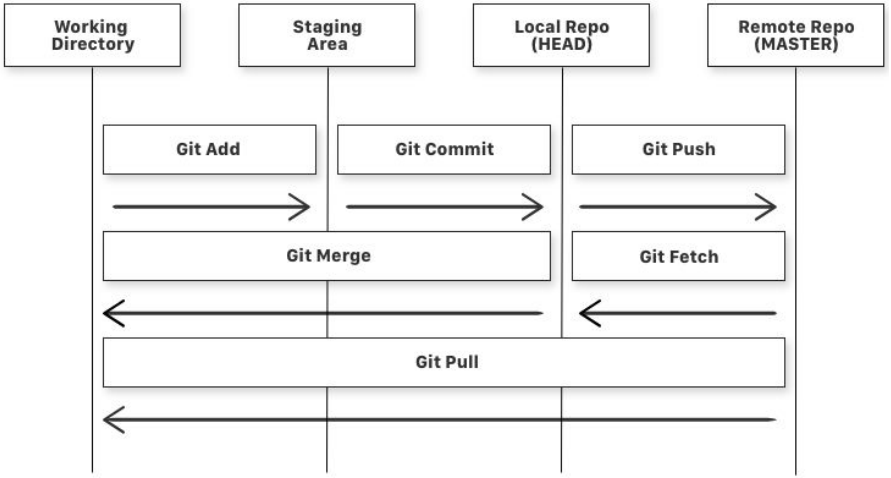
Se si considera un file nella Working Directory, questo può trovarsi in 3 stati possibili

1. **Untracked/Modified:** Il file con le modifiche non è ancora stato salvato nel local repository
2. **Staged:** Il file con le modifiche è pronto per essere committato nel local repository, ma non è ancora stato “committato”
3. **Committed:** Il file con le modifiche è salvato nel local repository

Lavorare con Git, quindi, consiste nel **creare/modificare un file, aggiungerlo alla staging area ed effettuare un commit**

Tuttavia, per poter collaborare è necessario sincronizzare le nostre modifiche con quelle del repository remoto e questo avviene “pushando” le nostre modifiche e “pullando” le modifiche effettuate dagli altri e risolvendo eventuali conflitti che si creano

Flusso di lavoro - completo



Vedremo come gestire al meglio questo flusso per renderlo il più fluido possibile

INSTALLARE GIT

Installare Git

macOS
https://git-scm.com/download/mac

Windows
https://git-scm.com/download/win

Linux
https://git-scm.com/download/linux

Installare Git

In un terminale digitate `git --version`. Se l'installazione è andata a buon fine dovreste visualizzare la versione di git appena installata

Impostare nome e email come identificativo scrivendo nel terminale:
`git config --global user.name "[firstname lastname]"`
`git config --global user.email "[valid-email]"`

Sempre da terminale create una chiave ssh sul vostro computer digitando
`ssh-keygen -t ed25519 -C "your_email@example.com"` e premete invio confermando ogni prompt senza inserire niente

Andate nella home (cartella utente) e visualizzate i file nascosti, lì dovreste trovare una cartella ".ssh". All'interno troverete il file "id_ed25519.pub" apritelo con un editor di testo (blocco note o VSCode)

Create un account GitHub

Seguite questa guida per aggiungere la chiave ssh a GitHub

Git GUI Client

Il miglior modo di utilizzare Git è da riga di comando con la Command Line Interface (CLI) il consiglio è di approfondire il tema per il futuro

Esistono un'infinità di GUI Client per Git (pagina ufficiale di Git) provatene un po' per vedere con quale vi trovate meglio, tra i più famosi possiamo trovare

GitKraken (a pagamento, free con GitHub for Educations)

SourceTree (Free open source)

GitHub Desktop (Free open source)

Per le operazioni più comuni è possibile usare anche Visual Studio Code

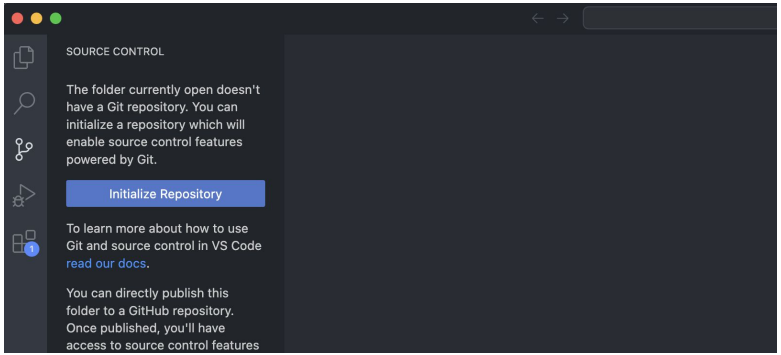
Git in VSCode

Inizializzare un repository

In Visual Studio Code è presente un'integrazione con Git. Nella maggior parte dei casi è consigliabile gestire tramite VS Code le operazioni più comuni così da non dover passare da un programma all'altro per effettuare delle modifiche

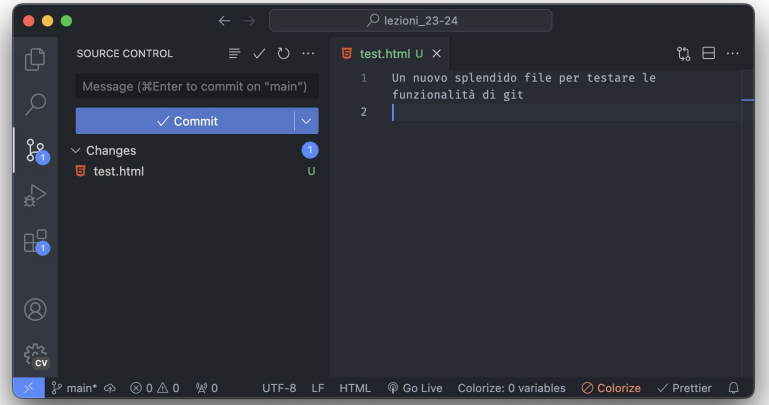
Per inizializzare un repository cliccate sull'icona di Git e create un nuovo repository

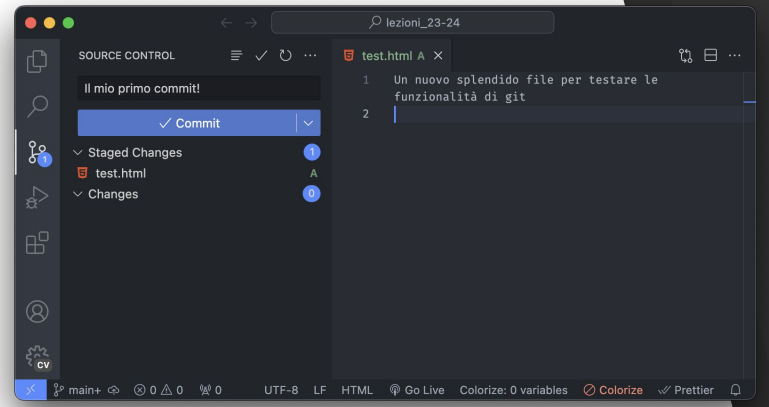
Da questo momento in poi ogni volta che effettuerete una modifica affianco all'icona di Git verrà visualizzato un badge con il numero di file con una modificata



Staging e Commit

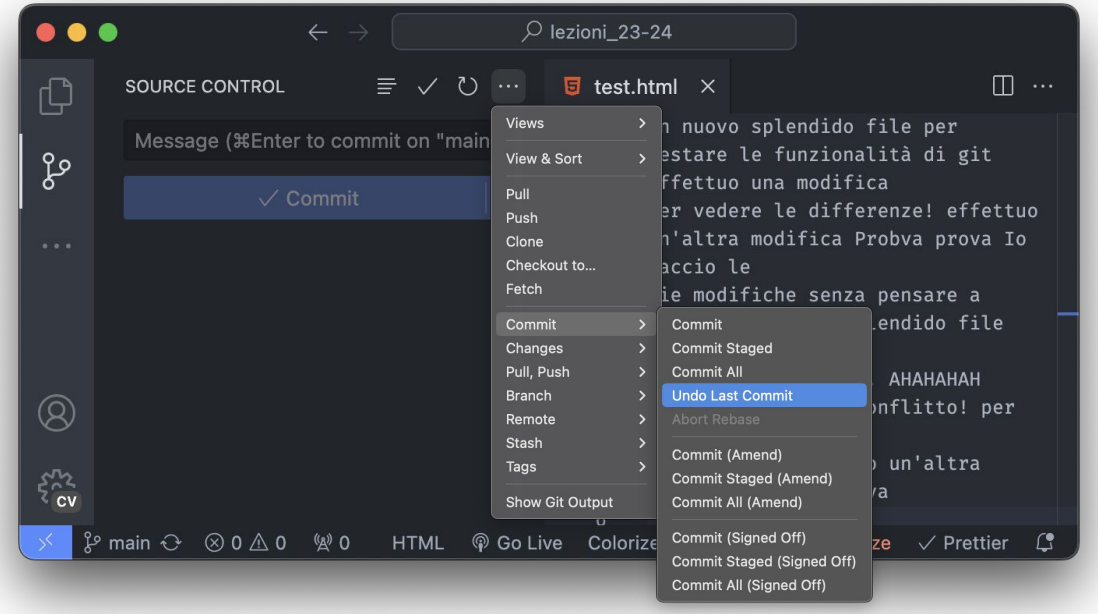
Tornando all’area dedicata a Git dopo una modifica è possibile aggiungere all’area di staging la modifica cliccando sul “+” affianco al nome del file ed effettuare un commit scrivendo il messaggio nel box di testo.





Annullare ultimo commit

Se si vuole annullare l'ultimo commit è possibile farlo selezionando puntini -> Commit -> Undo last Commit



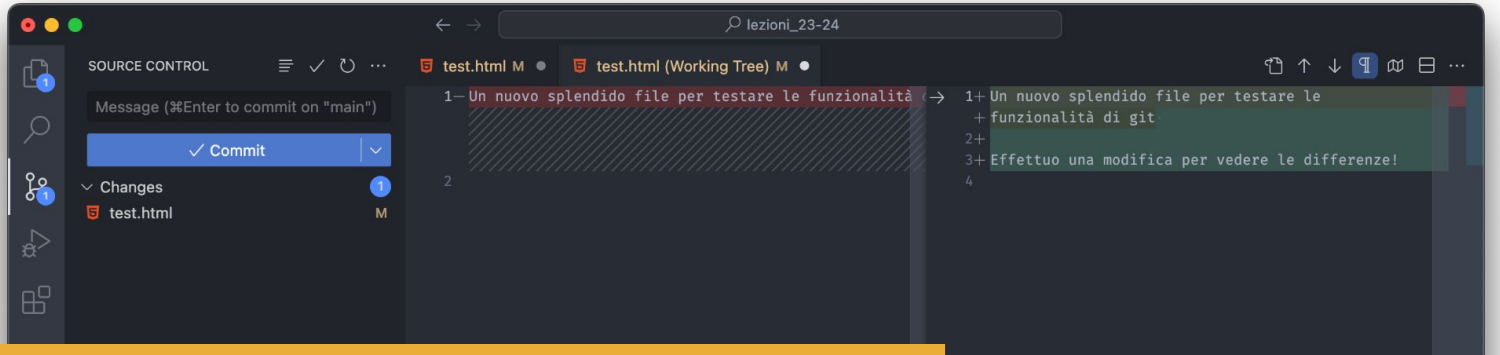
Diff e Restore

Passando sul nome del file con il mouse vengono visualizzate varie opzioni: “Open file”, “Discard changes” e “Stage changes”

Discard changes permette di annullare tutte le modifica all’ultimo stato salvato del file

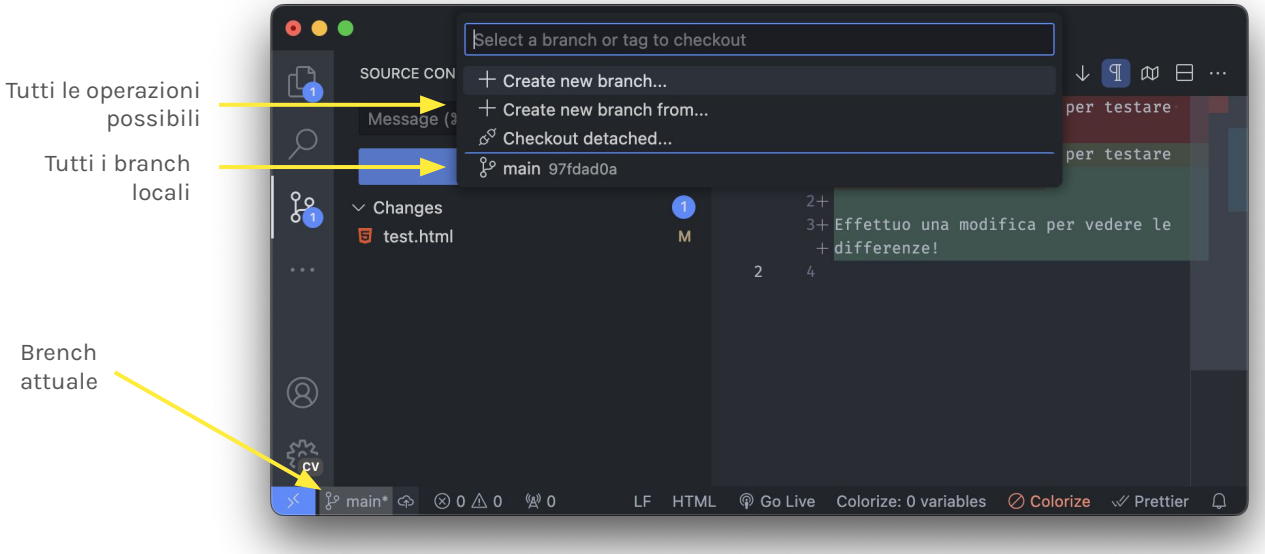
Per visualizzare le modifiche effettuate che verrebbero annullate, è possibile utilizzare “diff” cliccando sul nome del file

Nella schermata a sinistra si vede lo stato precedente del file, a destra le modifiche aggiunte



Branch

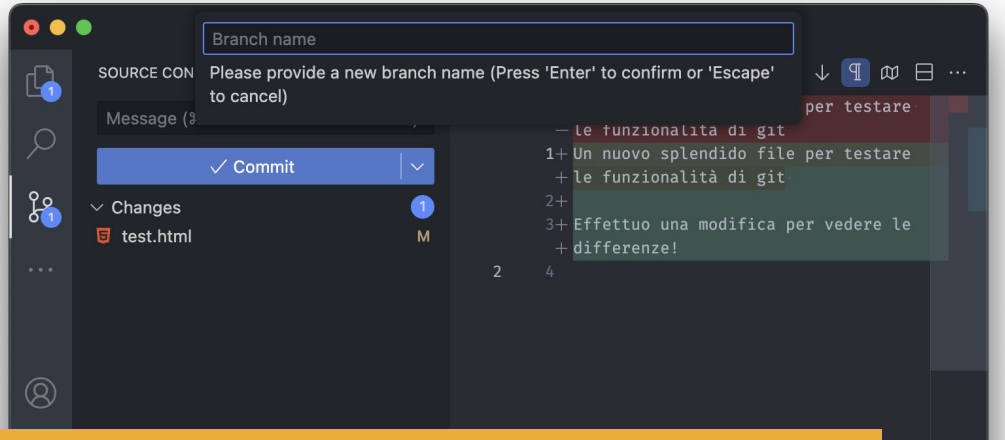
Per gestire i branch in VS Code esiste un pannello dedicato



New Branch

Selezionando “Create new Branch…” e inserendo il nome del nuovo Branch si creerà un nuovo Branch a partire dal Branch corrente. Per creare un nuovo Branch a partire da un Branch differente è possibile selezionare “Create new Branch from…”, selezionare il Branch di origine e quindi inserire il nome del nuovo Branch

In tutti i casi, dopo la creazione del nuovo Branch, VS Code effettuerà il Checkout automaticamente al nuovo Branch

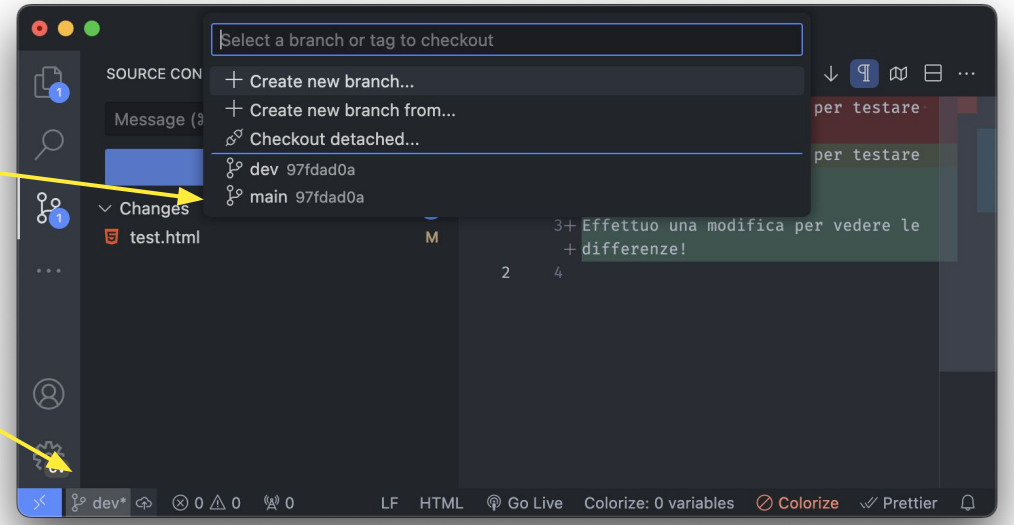


Checkout

Per passare da un Branch all'altro, selezionare il Branch corrente nella Status Bar e selezionare un Branch differente da quello corrente

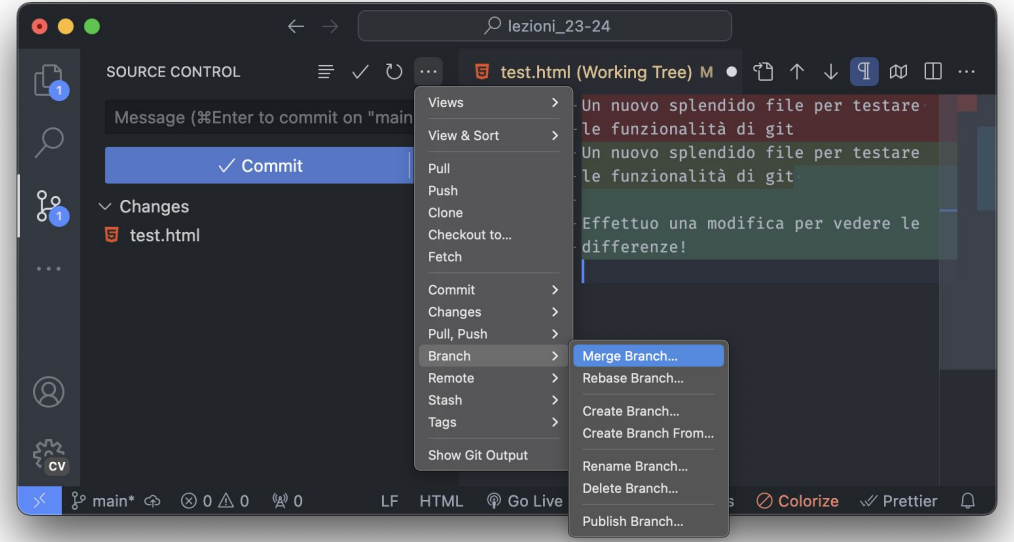
Possiamo selezionare "main"

Brench attuale è "dev"



Merge

Per riunificare due Branch è possibile effettuare un Merge. Posizionandosi sul Branch principale (cioè quello che deve ricevere le modifiche, e.g., “main”) selezionare i tre “puntini -> Branch -> Merge Branch…” quindi selezionare il Branch da cui prendere le modifiche



Merge

A seconda delle impostazioni è possibile sia necessario effettuare il commit del merge (salvare le modifiche derivanti dal merge). A volte è possibile che il merge avvenga in modalità “fast forward” e quindi sia trasparente nella history di Git

Una volta effettuato il Merge è possibile eliminare il Branch importato

Attenzione! Se il Branch esiste solo localmente, questa operazione è irreversibile. Prima di eliminare un Branch dovete essere sicuri che tutto sia andato a buon fine.

Git nel Cloud

Git e Github

Tutte le operazioni viste fino a ora sono relative all’uso di Git in locale. Per la collaborazione è necessario usare un repository centralizzato

Una delle alternative più diffuse è Github

Git e GitHub sono due concetti distinti, spesso confusi, ma complementari nell'ambito dello sviluppo del software. Git, come abbiamo visto è un DVCS (sistema di controllo delle versioni distribuito), mentre GitHub è una piattaforma remota che fornisce un'infrastruttura centralizzata per l'hosting e la collaborazione su progetti basati su Git. Potete immaginare Github come un Google Drive per i repository Git

Gli sviluppatori utilizzano GitHub per condividere, collaborare e gestire progetti basati su Git. GitHub offre funzionalità aggiuntive come tracciamento dei problemi (issues), richieste di pull (pull requests), e strumenti di collaborazione

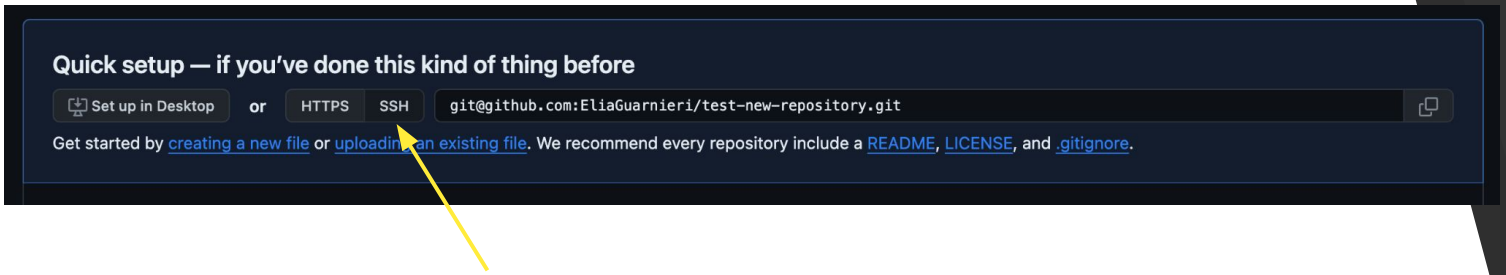
Esistono alternative altrettanto valide come BitBucket e GitLab, ma Github è di gran lunga quello più utilizzato nello sviluppo di progetti open source

Caricare il repository su Github

Su Github, nella scheda dei repository, selezionate “New” per creare un nuovo repository

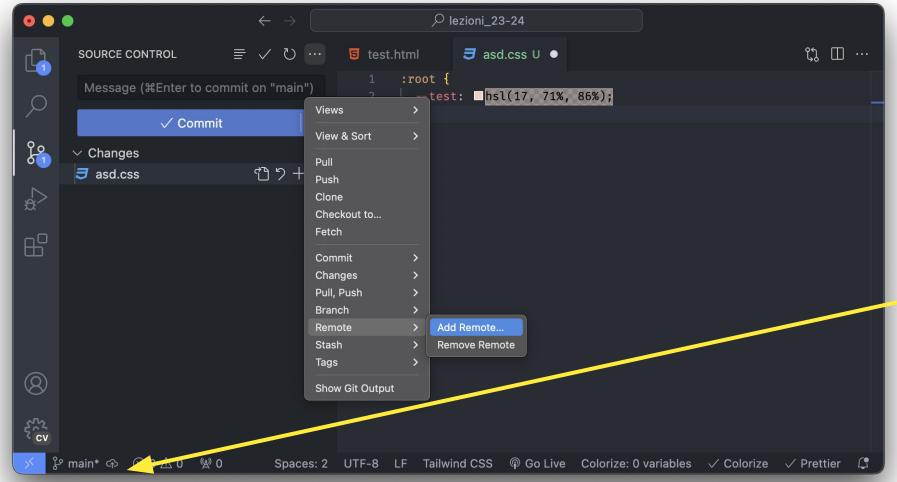
Scegliete un nome del repository e cliccate sul bottone “Create repository”

Nella pagina successiva alla scheda Quick setup selezionate “SSH” e copiate l’url



Caricare il repository su Github

In VS Code nella Status Bar selezionate puntini -> Remote -> Add Remote...

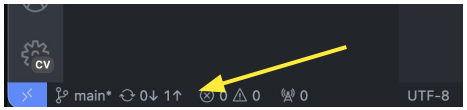


Icona con la nuvoletta rappresenta il comando per pubblicare il Branch sul remote

E inserite nel box di testo il link copiato su Github e date un nome al remote repository (per convenzione si usa origin, ma il nome è arbitrario). Infine, cliccate su “Publish Branch” per pubblicare il branch corrente. Da questo momento il repository locale è sincronizzato con il repository remoto su Github

Stato di sincronizzazione

È possibile verificare lo stato di sincronizzazione dalla Status Bar di VS Code.



In questo esempio abbiamo 0 modifiche da scaricare (Pull) e una modifica da caricare (Push)

Possiamo sincronizzare lo stato cliccando sull'indicatore. Questo farà Pull e Push in automatico e, salvo conflitti, sincronizzare i due repository

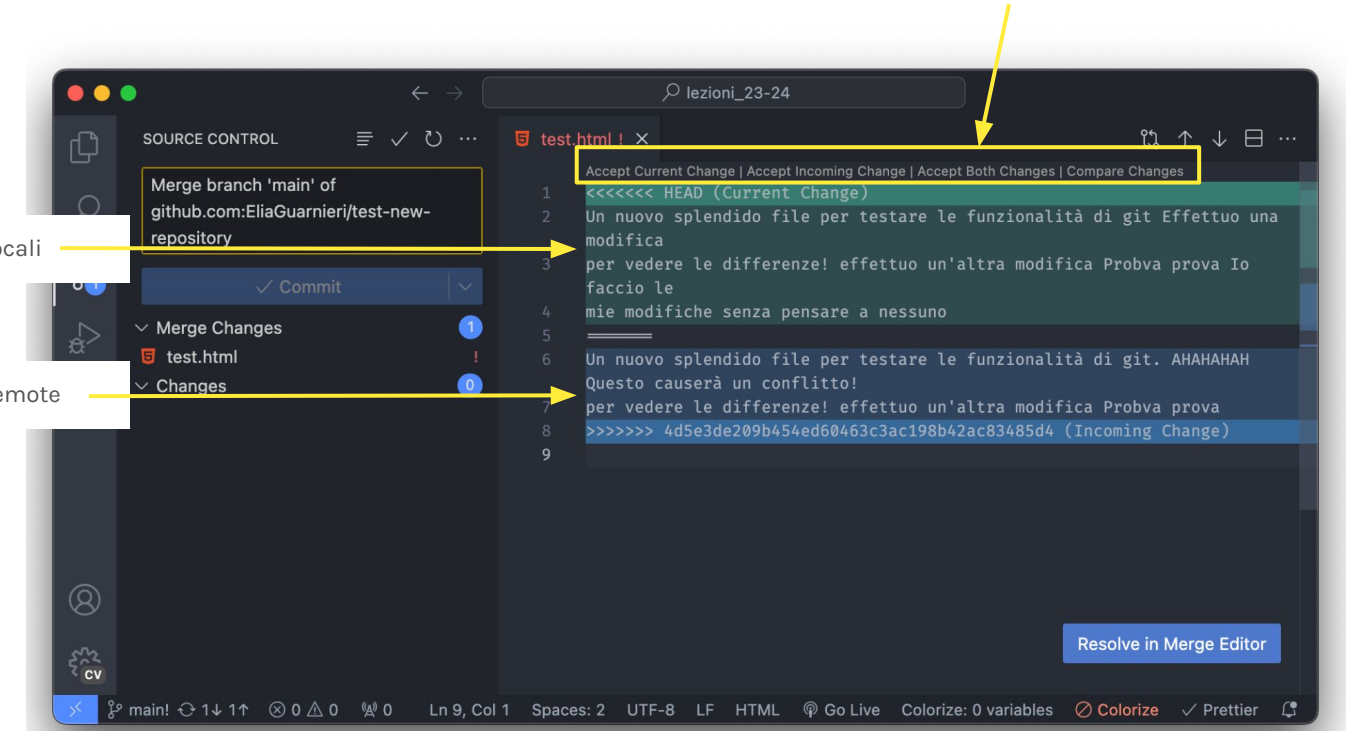
Risolvere conflitti di Merge

Se qualcuno effettua delle modifiche sullo stesso file alla stessa riga è possibile che Git non sia in grado di risolverlo autonomamente

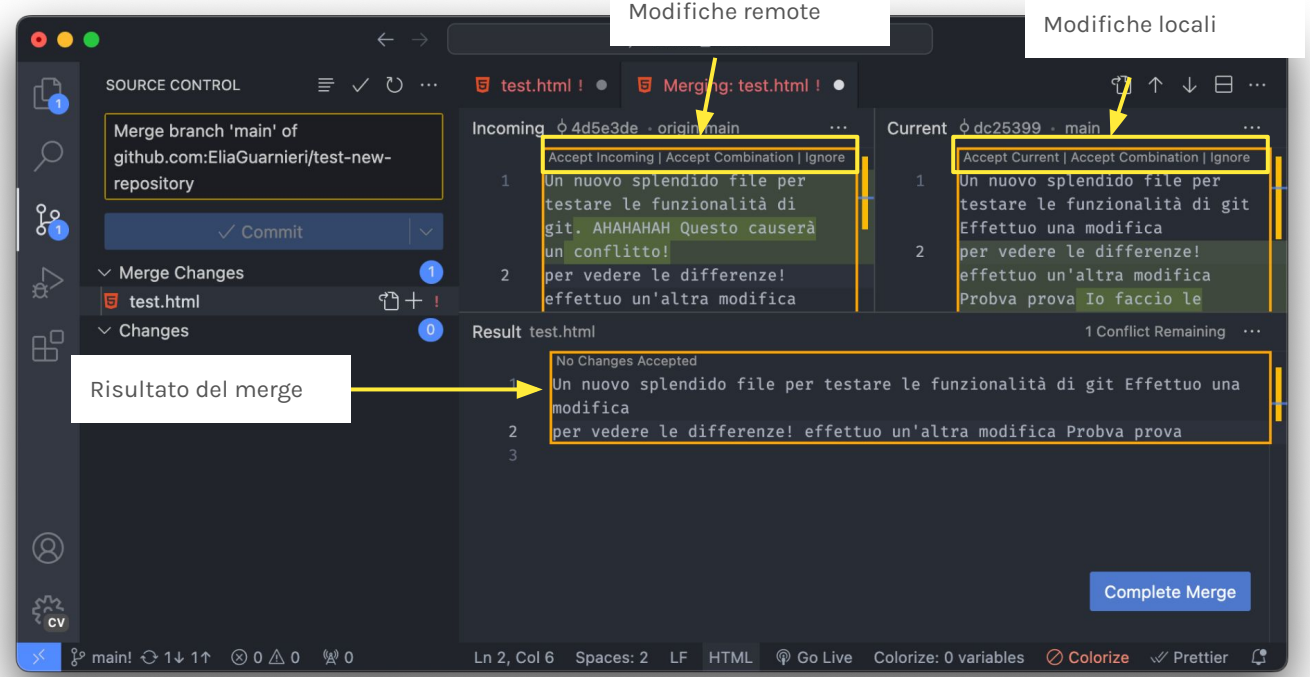
Nel momento in cui tenteremo di effettuare la sincronizzazione ci verrà segnalato che è necessario risolvere manualmente il conflitto

Esistono due modi per risolvere i conflitti

Risolvere inline



Risolvere nel Merge Editor

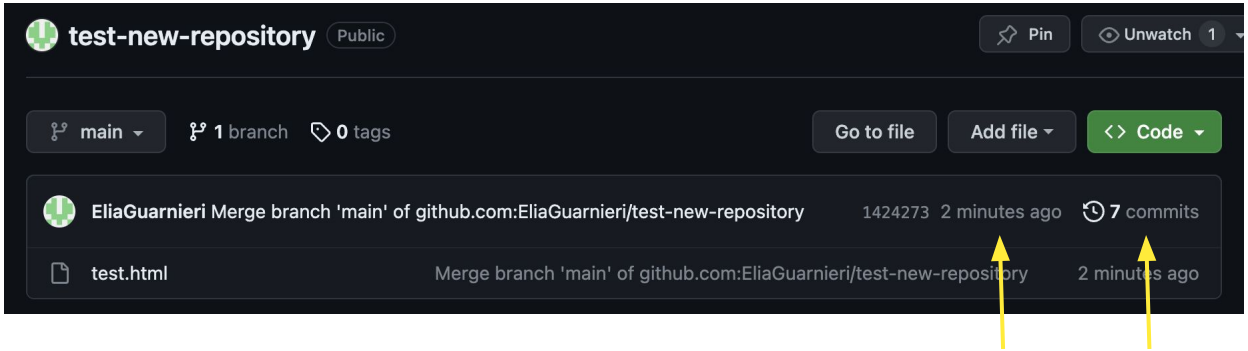


Risolvere nel Merge Editor

In entrambi i casi, una volta editato il file per risolvere il merge, sarà necessario fare un commit per confermare le modifiche. Per convenzione i messaggi del commit relativo al Merge contengono il Branch che sta subendo il merge. Il consiglio è di utilizzare il messaggio di default di VS Code

La history

Da VS Code non è possibile vedere la storia delle modifiche del repository, ma accedendo alla pagina di Github relativa al repository, è possibile vedere il numero di commit e le ultime modifiche



Cliccando sui commit è possibile visualizzare tutte le modifiche e tutta la storia relativa al branch

Un po’ di reference

Tutorial interattivo
https://learngitbranching.js.org/?locale=it_IT

VS Code e Git
https://code.visualstudio.com/docs/sourcecontrol/overview

Git cheat sheet
https://education.github.com/git-cheat-sheet-education.pdf



ESERCITAZIONE 8

FINE